



```

import A5O
import Closure
import Hops
set_option autoImplicit false

/-! # Hops — satisfiability witness (non-vacuity).
A concrete two-hop ledger: operator 0 appoints 1 to appoint 2 for action 0;
1 does so at time 1; 2 executes action 0 at time 2. Every hypothesis of
`Hops.every_execution_traces` is discharged on this model and the
conclusion `Traces 2 0 2` is derived. `not_root_link` shows the top link
is NOT a root link — the hop clause is genuinely used. -/

namespace HopsWitness
open Hops

abbrev Party := Nat
abbrev Action := Nat

def appointAct (A : Party) (X : Action) : Action := 100 + 10 * A + X
def Root (P : Party) : Prop := P = 0

def wC : Party → Party → Action → Nat → Prop := fun _ _ _ _ => True
def wI : Party → Prop := fun _ => True
def wM : Party → Action → Prop := fun _ _ => True
def wS : Party → Action → Prop := fun _ _ => True

/-- Grants on the ledger: (0 → 1, appoint 2 for 0) at 0; (1 → 2, action 0) at 1. -/
def wAppointed : Party → Party → Action → Nat → Prop := fun P A X t =>
  (P = 0 ∧ A = 1 ∧ X = appointAct 2 0) ∨ (P = 1 ∧ A = 2 ∧ X = 0 ∧ 2 ≤ t)

/-- Revocation-dominance stands for the root, and for 1's grant to 2 from t=1. -/
def wR : Party → Party → Action → Nat → Prop := fun P A X t =>
  P = 0 ∨ (P = 1 ∧ A = 2 ∧ X = 0 ∧ 2 ≤ t)

def wRevoke : Party → Party → Action → Nat → Prop := fun _ _ _ _ => False

/-- Executions: 1 performs the appointing act at 1; 2 performs action 0 at 2. -/
def wExecutes : Party → Action → Nat → Prop := fun A X t =>
  (A = 1 ∧ X = appointAct 2 0 ∧ t = 1) ∨ (A = 2 ∧ X = 0 ∧ t = 2)

def wInAoC : Action → Unit → Prop := fun _ _ => True

abbrev wO := 0 wC wI wM wS wR wAppointed

```

/-- The gate: enforces exactly O (v4's CanonicalMonitor shape). -/

def wSys : ADASystem Party Party Action Nat := ⟨fun P A X t => wO P A X t⟩

theorem w_ADA : achievesADA wC wI wM wS wR wAppointed wInAoC wSys :=
 fun _ _ _ _ _ h => h

theorem w_intercepts : ∀ (A : Party) (X : Action) (t : Nat) (d : Unit),
 wInAoC X d → wExecutes A X t → ∃ P : Party, wSys.enforces P A X t := by
 intro A X t _ hEx
 unfold wExecutes at hEx
 cases hEx with
 | inl h =>
 obtain ⟨hA, hX, ht⟩ := h
 subst hA; subst hX; subst ht
 exact ⟨0, trivial, trivial, trivial, trivial, Or.inl rfl, Or.inl ⟨rfl, rfl, rfl⟩⟩
 | inr h =>
 obtain ⟨hA, hX, ht⟩ := h
 subst hA; subst hX; subst ht
 exact ⟨1, trivial, trivial, trivial, trivial,
 Or.inr ⟨rfl, rfl, rfl, Nat.le_refl 2⟩, Or.inr ⟨rfl, rfl, rfl, Nat.le_refl 2⟩⟩

def wG : @GatedSystem Party Party Action Nat Unit wExecutes wInAoC := ⟨wSys,
 w_intercepts⟩

theorem w_H1 : AppointIsConsequential wInAoC appointAct := fun _ _ _ _ => trivial

theorem w_H2 : Origin wAppointed wExecutes appointAct Root := by
 intro P A X t hApp
 cases hApp with
 | inl h => exact Or.inl h.1
 | inr h =>
 obtain ⟨hP, hA, hX, ht⟩ := h
 subst hP; subst hA; subst hX
 exact Or.inr ⟨1, ht, Or.inl ⟨rfl, rfl, rfl⟩⟩

theorem w_H3 : Resolves wR wAppointed appointAct Root := by
 intro P A X t hR
 cases hR with
 | inl h => exact Or.inl h
 | inr h =>
 obtain ⟨hP, hA, hX, ht⟩ := h
 subst hP; subst hA; subst hX
 exact Or.inr ⟨0, 0, Nat.lt_of_lt_of_le (by decide) ht, Or.inl ⟨rfl, rfl, rfl⟩, Or.inl rfl⟩

```
theorem w_H4 : OneAppointer wAppointed := by
  intro Q Q' P Y s s' h h'
```

```
  cases h with
```

```
  | inl a =>
```

```
    cases h' with
```

```
    | inl b => exact a.1.trans b.1.symm
```

```
    | inr b =>
```

```
      -- a: P = 1, b: P = 2 — impossible
```

```
      have h1 : P = 1 := a.2.1
```

```
      have h2 : P = 2 := b.2.1
```

```
      subst h1; exact absurd h2 (by decide)
```

```
  | inr a =>
```

```
    cases h' with
```

```
    | inl b =>
```

```
      have h1 : P = 2 := a.2.1
```

```
      have h2 : P = 1 := b.2.1
```

```
      subst h1; exact absurd h2 (by decide)
```

```
    | inr b => exact a.1.trans b.1.symm
```

```
theorem w_H5 : RevokePersists wRevoke := fun _ _ _ _ _ h => h.elim
```

```
theorem w_P2 : P2 wR wAppointed wRevoke := fun _ _ _ _ _ h => h.elim
```

```
/-- THE WITNESS: 2's execution of action 0 at time 2 traces to root 0. -/
```

```
theorem two_hop_traces : Traces wC wI wM wS wR wAppointed appointAct Root 2 0 2 :=
  every_execution_traces wC wI wM wS wR wAppointed wExecutes wInAoC appointAct Root
  wG w_ADA w_H1 w_H2 2 2 0 () trivial (Or.inr ⟨rfl, rfl, rfl⟩)
```

```
/-- The hop clause is load-bearing: 0 did not appoint 2 directly, so no
root-link exists at the top. -/
```

```
theorem not_root_link : ¬ wO 0 2 0 2 := by
```

```
  intro h
```

```
  have hApp : wAppointed 0 2 0 2 := h.2.2.2.2.2
```

```
  cases hApp with
```

```
  | inl a => exact absurd a.2.1 (by decide)
```

```
  | inr a => exact absurd a.1 (by decide)
```

```
/-- Every hypothesis of the cascade theorem is also satisfiable here
(revocation empty, so P2/H5 hold trivially); the root has been shown to
be the only origin. -/
```

```
theorem w_CFunctional_fails : ¬ CFunctional wC :=
```

```
  fun h => absurd (h 0 1 0 0 0 trivial trivial) (by decide)
```

```
end HopsWitness
```

```
#print axioms HopsWitness.two_hop_traces  
#print axioms HopsWitness.not_root_link  
#print axioms HopsWitness.w_H3  
#print axioms HopsWitness.w_H4
```